DATA STRUCTURE

LAB PROGRAMS

15. Splay Tree in C (Insert, Search, Inorder)

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int key;
    struct Node *left;
    struct Node *right;
};

struct Node* newNode(int key)
{
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->key = key;
    node->left = NULL;
    node->right = NULL;
    return node;
}

struct Node* rightRotate(struct Node* x)
{
    struct Node* y = x->left;
    x->left = y->right;
    y->right = x;
```

```
    return y;
}

struct Node* leftRotate(struct Node* x)
{
    struct Node* y = x->right;
    x->right = y->left;
    y->left = x;
    return y;
}

struct Node* splay(struct Node* root, int key)
{
    if(root == NULL || root->key == key)
        return root;

    if(key < root->key)
    {
        if(root->left == NULL)
            return root;

        if(key < root->left->key)
        {
            root->left->left = splay(root->left->left, key);
            root = rightRotate(root);
        }
        else if(key > root->left->key)
        {
            root->left->right = splay(root->left->right, key);
```

```
        if(root->left->right != NULL)
            root->left = leftRotate(root->left);
        }

        return (root->left == NULL) ? root : rightRotate(root);
    }
    else
    {
        if(root->right == NULL)
            return root;

        if(key > root->right->key)
        {
            root->right->right = splay(root->right->right, key);
            root = leftRotate(root);
        }
        else if(key < root->right->key)
        {
            root->right->left = splay(root->right->left, key);

            if(root->right->left != NULL)
                root->right = rightRotate(root->right);
        }

        return (root->right == NULL) ? root : leftRotate(root);
    }
}

struct Node* insert(struct Node* root, int key)
{
```

```
    if(root == NULL)
        return newNode(key);

    root = splay(root, key);

    if(root->key == key)
        return root;

    struct Node* node = newNode(key);

    if(key < root->key)
    {
        node->right = root;
        node->left = root->left;
        root->left = NULL;
    }
    else
    {
        node->left = root;
        node->right = root->right;
        root->right = NULL;
    }

    return node;
}

void inorder(struct Node* root)
{
    if(root != NULL)
    {
```

```
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

int main()
{
    struct Node* root = NULL;
    int n, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &key);
        root = insert(root, key);
    }

    printf("Inorder Traversal: ");
    inorder(root);

    printf("\nEnter element to search: ");
    scanf("%d", &key);

    root = splay(root, key);

    if(root != NULL && root->key == key)
```

```
        printf("Element %d Found\n", key);
    else
        printf("Element %d Not Found\n", key);

    if(root != NULL)
        printf("Root after search: %d\n", root->key);

    return 0;
}
```

Output

```
Enter number of elements: 7
Enter elements:
10 20 30 40 50 60 70
Inorder Traversal: 10 20 30 40 50 60 70
Enter element to search: 65
Element 65 Not Found
Root after search: 60


=== Code Execution Successful ===
```